

HAKING

WORKSHOPS

HACK WITH PYTHON



RUI SILVA



TABLE OF CONTENTS

3	Module 1 Introduction
5	Python as an interpreted language: How to choose correct interpreter, install it, run it
10	Standards and batteries included: Standards and PEP8
13	What you learned
14	Module 2 Python Basics
14	Python data types and flow control statements
16	Lists (slices), dictionaries (loop over items), sets
19	Python Internals
28	What you learned

29 | Module 3 Files

29 Files

30 Opening and reading from files

33 Practical exercise

40 What you learned

41 | Module 4 Project

41 Get information from API

45 Data transformation

46 Plotting the data

50 What you learned

MODULE 1

INTRODUCTION

WHY PYTHON?

INTRODUCTION ABOUT PYTHON PROGRAMMING LANGUAGE

Python was created by Guido Van Rossum in the Netherlands. The language itself was created to be as simple as possible to read and use. Guido is still a huge part of Python and he is known in the community as the benevolent dictator for life (BDFL). The Python philosophy is summarised in the [Zen of Python](#), a collection of guidelines that every Python core developer follows and every Python programmer should follow. The most important ones for people starting with Python are:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Readability counts.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.

Python is a language that is gaining a lot of [popularity as a learning language](#) because of its ease of use and its “batteries included” philosophy. These batteries mean that any Python standard distribution comes with a very powerful set of libraries to help overcome common simple problem.

For example, http connectors, sqlite (a relational database), csv readers, a simple http server (actually called SimpleHTTPServer) amongst many other.

The current version of Python is 3.4. But for this workshop we'll use version 2.7 which is in FreeBSD's repositories. Everything we do here also applies to python 3.4.

Python is a very well documented language. You can check the [official documentation](#) that has every information you will need and also many tutorials you can try out.

LEARNING THE STRENGTHS OF THE LANGUAGE AND WHAT'S GOOD WITH PYTHON

Python has a lot of strengths that make it a great language to develop with. I will list some of those strengths and explain.

First of all is the readability. Python was written to be easy to read. There's a great insight from Guido Van Rossum which states that code is read much more often than its written so readability is also much more important than ease of writing.

Interpreted language for scripting. Being an interpreted language gives python the benefit of being great for scripting. This means that you can write a file with code and just run the file without compiling (Python's internals take care of that for you). We will learn how to do this later on.

OBJECT ORIENTATION

In Python everything is an object. This makes the language very extensible, for example, you can extend python's list type to have some special behavior for your project.

OPEN SOURCE

Because Python is completely Open Source you can inspect every source file (as long as it isn't a C extension) and see what code is being run. This might not seem like a big deal but when you're getting strange bugs it's a huge benefit to be able to look at the code causing it.

For all of the reasons above and more, Python is a great language to develop with because it's mainly easy to learn. This makes Python perfect for scripting, web development (Django, Pylons, Flask...), systems administration (ansible...).

LEARNING WHERE TO USE PYTHON AND WHY

Python can be used to do anything. But it does have some drawbacks. One of those is performance. Python is not the most performant language. All that readability and ease of use comes with a price. But it is fast enough. We don't need a language to be super optimised to do web development. Nowadays most scripts or projects have to connect somewhere to get extra information and use it. Getting that information will be your bottleneck. Network connections are always the slowest part of a piece of code.

PYTHON AS AN INTERPRETED LANGUAGE

HOW TO CHOOSE CORRECT INTERPRETER, INSTALL IT, RUN IT

As mentioned before for this workshop we'll be using Python 2.7. To install Python on a FreeBSD machine type on the console as root:

```
$ pkg install python
```

After installing python you'll have access to the interpreter. Just type python on a console terminal and you'll get a python shell.

This shell is your interface to python, with it you can do anything that python can do. Let's try printing something back:

```
>>> print "Hello World"
```

```
Hello World
```

As a side note: if you see `print` being used with parentheses, like `print("hello")` don't worry, it also works but only in version 3.0 that's the only way that works (`print` without parentheses was removed in python 3.0). This python shell is called the interpreter. In this shell you can do anything with python. Let's try a more advanced example. Open a python shell (just type `python` in the terminal) and try the following commands:

```
>>> print "1 + 1 is", 1 + 1
```

```
1 + 1 is 2
```

As you can see python also does simple math operations as you would expect. Let's try importing a module so we can use some advanced features.

```
>>> from math import sin
```

```
>>> sin(1)
```

```
0.8414709848078965
```

Here we are importing the `sin` function from the `math` module. A more verbose way of reading the import line could be: from the `math` module import the `sin` function.

Now that you know how to use the shell try running the following line: `import this`. What do you get?

PYTHON VIRTUAL ENVIRONMENTS

Python has a particular way of doing imports. Let's take the `from math import sin` line from the example before. Where does the `math` module come from? The `math` module comes from the python standard library. When you try to import something python will try several places starting with the local folder you are on and then going through the `PYTHONPATH` environment variable (if you don't know what that is just think of it as a list of folders for python to search through). In this course we'll be installing some third party python packages and we'll be using a python package installation tool called `pip`. By default this tool installs the packages system wide. This means that anyone that opens a python shell would have the package available, which might not seem like a bad thing but there are some drawbacks. This would lead to problem down the line because you are only allowed one version of each package in your system and if 2 users needed 2 different version this wouldn't work.

This is where virtual environments come in. Basically we are going to be creating an environment per project we do. This way we'll be able to install everything we need without interfering with the system.

Let's get started.

First we need to install the virtual environment package. Let's do that using pkg.

```
$ pkg install py27-virtualenv
```

Now that virtual environment is installed let's initialise one for this course.

```
$ virtualenv python-workshop  
  
New python executable in python-workshop/bin/python2.7  
  
Also creating executable in python-workshop/bin/python  
  
Installing setuptools, pip...done.
```

Now that our virtual environment is installed let's activate it to make sure we are using the new python "installation" (it's not really a whole new installation, it's just some symbolic links to the system wide python).

```
$ which python  
  
/usr/local/bin/python  
  
$ cd python-workshop  
  
$ source bin/activate  
  
(python-workshop)$ which python  
/usr/home/vagrant/python-workshop/bin/python
```

Notice how after activating the virtualenv the command prompt changed and the python used also changed? When your virtual environment is activated you can install any package without messing up the system. To deactivate the virtual environment you can just type deactivate in the console.

Try installing the requests package for python. With the virtual environment activated just type:

```
$ pip install requests

Downloading/unpacking requests

Downloading requests-2.4.3-py2.py3-none-any.whl (459kB) : 459kB downloaded

Installing collected packages: requests

Successfully installed requests

Cleaning up...
```

And that's it. The [requests](#) package is installed and ready to be used.

Let's test it to make sure it works. Open a python shell and try:

```
>>> import requests >>> response = requests.get("https://google.com/") >>>
response.content
```

This piece of code will get the HTML code for “https://google.com/”. The response content will have a lot of javascript too. This particular library is very useful to make requests to API's throughout the internet. For example: if you do a get to “http://ip-api.com/json” it will tell you what your current IP is and where you are. Try it out.

TEXT EDITOR (KATE, GEDIT, BRACKETS)

To write code in Python we can use very advanced tools like [IDE's](#). But for now like stick to the simple ones, text editors. A text editor is an application that allows you to write text files. There are a lot of then you can choose from. If you are working on a shell you need to use some shell text editors like [vim](#) or [nano](#). Although these editors are very powerful they do have a very steep learning curve and are not the most friendly to newcomers.

If you are just starting I advise you to use a GUI text editor like [brackets](#), [kate](#) or [gedit](#). These editor have code highlighting and support plugins if you are feeling adventurous. For example, in brackets if you install the “integrated development” plugin you’ll be able to run the python scripts you create right from the editor.

Get comfortable with your text editor of choice and try to learn as much keyboard shortcuts as possible as the editor you'll be your friend.

HOW TO CREATE HELLO WORLD, FROM INTERPRETER AND WITH .PY SCRIPT

Now let's try creating a python script and run it in our interpreter. Open your favourite text editor and create a file named `hello.py` in your virtual environment directory with the following content:

```
print "Hello world"
```

Save the file and run:

```
$ python hello.py
```

```
Hello World
```

This is how easy it is to write script files in python. Let's create a more complex one just to show off some of python. Let's use the requests library we installed in the previous section. Create a new file called `secondhello.py` and write the following:

```
import requests

print "What IP do you want to investigate?"

ip = raw_input()

response = requests.get("http://ip-api.com/json/" + ip)

print response.json()
```

Save the file. Try running the code. It will ask you for some IP, grab [your own](#) and paste it to the script when it asks for it.

STANDARDS AND BATTERIES INCLUDED

STANDARDS AND PEP8

Python is a standards language. Every feature that is included in python has a [PEP \(Python Enhancement Proposals\)](#) associated with it. These PEP's are what guide the language when developing new features. Remember the Zen of Python from before? It is defined in [PEP20](#). There is one particular PEP which everyone should be familiar with which is [PEP8](#). This PEP defines a standard for python code styling. With this everyone should have the same code style when writing python. This will make everyones code much easier to read and understand. It's a very good practice to know and follow this PEP when writing Python.

BATTERIES INCLUDED

When Python is mentioned as having "batteries included" this means that Python's standard library has a lot of modules that do some advanced things without having to install anything extra. Some examples of those modules are:

- urllib - this module is used to make HTTP requests
- csv - module used to read/write csv files
- math - advanced mathematical functions
- zlib - file compression compatible with gzip
- sqlite3 - relational database connector for [sqlite](#)
- pdb - the python debugger
- unittest - as the name states it's a unit testing framework

These are just some examples of what comes with python. You can check the full list at <https://docs.python.org/2/library/>.

Here is a sample script that uses some advanced “batteries” from python’s standard library.

```
import csv
from zlib import compress
from urllib2 import urlopen

# fetch content of url
sample_csv_url =
"https://raw.githubusercontent.com/pedroma/python-workshop/master/
Sacramentorealestatetransactionowns.csv"
response = urlopen(sample_csv_url)

# save content into new file locally
f = open("SacramentoRealEstate.csv", "w")
f.write(response.read())
f.close()

# read file we just saved
f = open("SacramentoRealEstate.csv", "rU")
csv_reader = csv.DictReader(f)

# do some dummy calculations
n_lines = 0
n_beds = 0
for line in csv_reader:
    n_beds += int(line["beds"])
    n_lines += 1

f.close()

# and print out result
print("There were {0} in Sacramento's {1} real estate
transactions.".format(n_beds, n_lines))
```

```
# now that we got what we need let's gzip the file

f = open("SacramentoRealEstate.csv", "rU")
gzipped_file_content = compress(f.read())
f.close()

# save the gzipped content into a new file
gzipped_file = open("SacramentoRealEstate.csv.gz", "w")
gzipped_file.write(gzipped_file_content)
gzipped_file.close()
```

The comments in the code should help you understand what's happening. I'll summarised here. First we fetch a CSV file from the internet. We save that file to the local machine (in the current folder). After that we read the file we just saved and feed it to the csv reader to be used for calculations. We iterate through the file's content and print out calculated information (the information used here is dummy).

After all calculations are done we want to save the file in a compress state so that it doesn't waste space. For that we read the file's content, feed it to the compress function and save the output into a new file. The core libraries we used in this script were: urllib, zlib and csv.

As you can see with 20 lines of actual code we are accessing the web, reading and writing various files and doing some minor calculations. Not bad for 30 lines.

Don't worry if you don't understand all of the code. This is just to show you what's possible with Python.

WHAT YOU LEARNED

THIS IS WHAT YOU LEARNED IN THIS MODULE:

- python's philosophy
- how to install python
- first contact with the python interpreter
- how python imports work
- working on a virtual environment
- creating and running a python script
- using text editors for coding
- standards and PEP8
- various modules which python comes bundled with

MODULE 2

PYTHON BASICS

PYTHON DATA TYPES AND FLOW CONTROL STATEMENTS

We learned some advanced concepts in Module 1. But let's retrace a little bit and consolidate our knowledge on the Python Programming language. Let's start with basic control flow statements.

IF / ELIF / ELSE

The first statement will be the if/elif/else. This is the most basic control flow statement you can have in Python. Here are some examples of other ways you can use the if statement. First let's start with the full description of the statement.

```
if condition:
    statement
elif other_condition:
    statement
....
else:
    statement
```

Let's try and check if some number is even. The way to do it would be:

```
if number % 2 == 0:
    is_even = True
else:
    is_even = False
```

Another way of writing this could be:

HAKING

```
is_even = False
if number % 2 == 0:
    is_even = True
```

We could even do:

```
is_even = True if number % 2 == 0 else False
```

Or even simpler:

```
is_even = number % 2 == 0
```

All these lines are equivalent. They achieve exactly the same goal which is to check if a number is even. Of course this is a very simple example but you can do things like:

```
processed_articles = slow_process(articles) if len(articles) < 10 else
quick_process(articles)
```

FOR LOOPS

Lets move onto the more interesting control flow statement which are fors and whiles. These are the statements that allow us to loop through iterables. An iterable is some object in python that you can iterate through, like lists or dictionaries. The for loop syntax is:

```
for value in iterable:
    statement
```

Simple right? Lets try doing something more interesting with a list. We'll need the squared value for every natural number till 10.

```
squared = []
for n in range(10):
    squared.append(n**2)
```

The results for this operation will be [0, 1, 4, 9, 16, 25, 36, 49, 64, 81] as expected. The range function is a great way of iterating through natural numbers. Another way of doing this loop could be:

```
squared = [n**2 for n i range(10)]
```

This is called a list comprehension and although we are not covering them just know that they are great and usually better have performance than for loops (when they can be used).

WHILE LOOPS

The while loop is much like the for but you use a condition to decide whether to leave the loop or not. You read the while exactly like its written: execute statement while condition is True.

```
while condition:  
    statement
```

While loops are not that used compared to for loops so just keep in mind that they exist.

LISTS (SLICES), DICTIONARIES (LOOP OVER ITEMS), SETS

LISTS

Python has some very interesting and easy to use built-in data types. Lists are one of those and probably the one that you'll be using the most. To create a list you just have to do `articles = []`. This creates an empty list ready to be used. Lists do not require that every element be the of the same type. Any list can have numbers, strings, other lists, tuples, dictionaries,... The most basic operations you can do with a list are:

- `append` - add an element to the list
- `remove` - remove an element from the list
- `index access` - you can access any position on the list by index number (`articles[3]`)
- `list slice` - this is a very useful list operation that we'll go into more detail

Above I mentioned tuples. Tuples are basically a fixed size list. They can be used when you know that your list will always have the same size. This is good because tuples perform better than lists (but with the mentioned size restrictions).

LIST SLICES

Whenever you have a list you might want to get a subset of it, or even access the last element. This is where list slicing comes into play. If you have a list with numbers from 0 to 10 but you only want the values from the 5th position to the last you can do:

```
>>> numbers = range(10)
>>> numbers
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> numbers[5:]
[5, 6, 7, 8, 9]
```

The same way you can get the element from the 5th position on you can also get the first 5 elements:

```
>>> numbers[:5]
[0, 1, 2, 3, 4]
```

This is very useful when manipulating lists. Now lets imagine you need to know what the last element of the list is. Usually you would do:

```
>>> numbers[len(numbers) - 1]
9
```

But python allows you to do:

```
>>> numbers[-1]
9
```

This mean that you can loop through your list in reverse order by making the indexes sis means that you can loop through the elements of your list in reverse. To do that you just start at -1 of the list all the way to the size of the list.

DICTIONARIES

Dictionaries are another very used data type from Python. This is simply a hash table. Instead of having a number defining where your element is you have a key. This key can be a lot of things, a number, a string even datetime object. For example:

```
>>> articles = {  
... 123: "Article 1",  
... 124: "Article 2"  
...}  
>>> articles[123]  
Article 1
```

In this example we are indexing each article to their unique identifier. If this was a list the keys would have to be integers starting at zero (unless you wanted to fill all the indexes from 0 to 122 with None to be able to access the article with articles[123]).

There are some ways of looping through dictionaries. I'll focus on the most common.

```
>>> for key in articles:  
... print key, articles[key]  
123 Article 1  
124 Article 2
```

As you can see iterating through the dictionary will iterate through its keys. You can then access the dictionary using the key. Another way to make this easier to read could be:

```
>>> for key, value in articles.items():  
... print key, value  
123 Article 1  
124 Article 2
```

In this case we just use the items() method to create a list of key / value pairs so that the for loop can iterate through.

One thing you must keep in mind is that dictionaries are not ordered. This means that you cannot expect the loop to go through the dictionary in the same order that you added the elements. Python does have an OrderedDict which makes the dictionary keep ordering but it performs worse than the normal dictionaries.

We will be using dictionaries later on this module and you'll have time to get familiar with them.

PYTHON INTERNALS

CLASSES AND OBJECT INSTANCES

Python is an Object-Oriented language. And as such it deals with classes and class instances. If you are not familiar with Object oriented programming that's ok but there might be some definitions you might not know (like instance or instance variable). But I'll try to explain as we go. A class is an object that wraps a behavior. In the last model we had the following code:

```
<style="text-align: justify;" align="LEFT">
sample_csv_url =
"https://raw.githubusercontent.com/pedroma/python-workshop/master/
Sacramentorealestatetransactions.csv"
response = urlopen(sample_csv_url)

f = open("SacramentoRealEstate.csv", "w")
f.write(response.read())
f.close()
```

The `open()` function returns a File object. The object is a File class instance and you can call methods on it (like the `write()` method in the line after). Note that I called `write()` a method and not a function. This is because `write()` is a function that belongs to the File object and works within its context. If you just called `write()` without the File instance it wouldn't work because it needs to know where to write to.

Lets create a simple class and try this for ourselves.

```
class Point(object):
    x = 0
    y = 0
```

After defining this class we can instantiate it (create an instance of this class by doing):

```
>>> p = Point()
>>> p.x
0
>>> p.y
0
```

Right now we have a Point with coordinates (0,0). But there's nothing we can do to change it. Let's add some code that will allow us to create an instance with different values for x and y.

```
class Point(object):
    x = 0
    y = 0
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y
>>> p = Point(x=3, y=5)
>>> p.x
3
>>> p.y
5
```

Good start. Let's try to understand what happened here. We defined a method inside Point. This is a special method. The `__init__` method is for class initialization and allows us to do some processing when a new instance is created. Our goal was to be able to set different values for x and y and it worked. The self parameter you see is so that we can access the current class and assign values to it. In other languages you don't usually see this parameter (it is assumed to be there) but Python has the "explicit is better than implicit" philosophy.

EVERYTHING IS AN OBJECT (DOCS STRINGS, GETTERS, SETTERS, OVERRIDE)

In Python everything is an object. Function, modules, even integers and built-in functions. Here is an example to show what I mean.

```
>>> i = 3
>>> i.bit_length()
2
```

If you want to know what a function does you can ask the function for its `__doc__` attribute which returns the documentation (provided the function has any). Let's ask python what the `bit_length` function does.

```
>>> i = 3
>>> print i.bit_length.__doc__
    int.bit_length() -> int

    Number of bits necessary to represent self in binary.
>>> bin(37)
'0b100101'
>>> (37).bit_length()
6
```

As you can see the `bit_length` function returns how many bits are needed to represent a number in binary. Most of python core functions are documented to help you when using them.

This “everything is an object” approach is very valuable and provides us with some very powerful features. Let's try doing something more advanced. Let's play with getters.

A getter is a special method that allows you to get an item from an object.

Let's improve python's dictionaries. Initialize a dictionary and try to access a key that we haven't inserted yet:

```
>>> articles = {}
>>> articles["article1"]
-----
KeyError                                Traceback (most recent call last)
<ipython-input-10-59d0a8c1d287> in <module>()
----> 1 articles["article1"]

KeyError: 'article1'
```

When we try to do this we get an exception. Let's create a dictionary that's doesn't throw one when we access a non-existent key. Because everything is an object we can override Python's dictionary object and provide our own behavior. For example:

```
class MyDict(dict):  
    def __getitem__(self, key):  
        if key not in self:  
            return None  
        return super(MyDict, self).__getitem__(key)  
  
>>> articles = MyDict()  
>>> articles["article1"]
```

As you can see now we don't get any problem when accessing a key that doesn't exist. Of course this is a very normal thing to want to do in Python and there is a shortcut for it.

```
>>> articles = {}  
>>> articles.get("article1")
```

The get method from a dictionary is a “get or return None” (alternatively you can pass a second argument to replace the None (for example: “if you can't find the value return the string “No value”).

This is a very powerful feature because it allows you to do some very advanced stuff. For example applying this to lists you can build a spreadsheet where you do sheet[“A1”] and internally you parse the “A1” to be the positions [0][0] on your list. You can check an example of that [here](#).

Now let's play with setters. They follow the same rules as getters but for setting values. Let's imagine that for some reason when we add a number to a dictionary we want to increment it by one. You could do it this way.

```
class MyDict(dict):  
    def __setitem__(self, key, value):  
        if type(value) == int:  
            value += 1  
        super(MyDict, self).__setitem__(key, value)  
  
>>> articles = MyDict()  
>>> articles["article"] = 1  
>>> articles["article"] = 2
```


As you can see we tried to store a 1 but we stored a 2. This is an advanced feature and allows us to understand how python works internally so we can take better advantage of the language.

PRACTICAL EXAMPLE

Let's do a slightly bigger example by using Twitter's API to get tweets. Twitter's API is a great source of information we easy enough to use that we can take advantage of it for this workshop. The main wall to access Twitter is the Authentication system (OAuth2). We'll be using a Python package to make our work easier called Tweepy. Let's start off creating a virtual environment (or use the one created in the first module) and install the package tweepy.

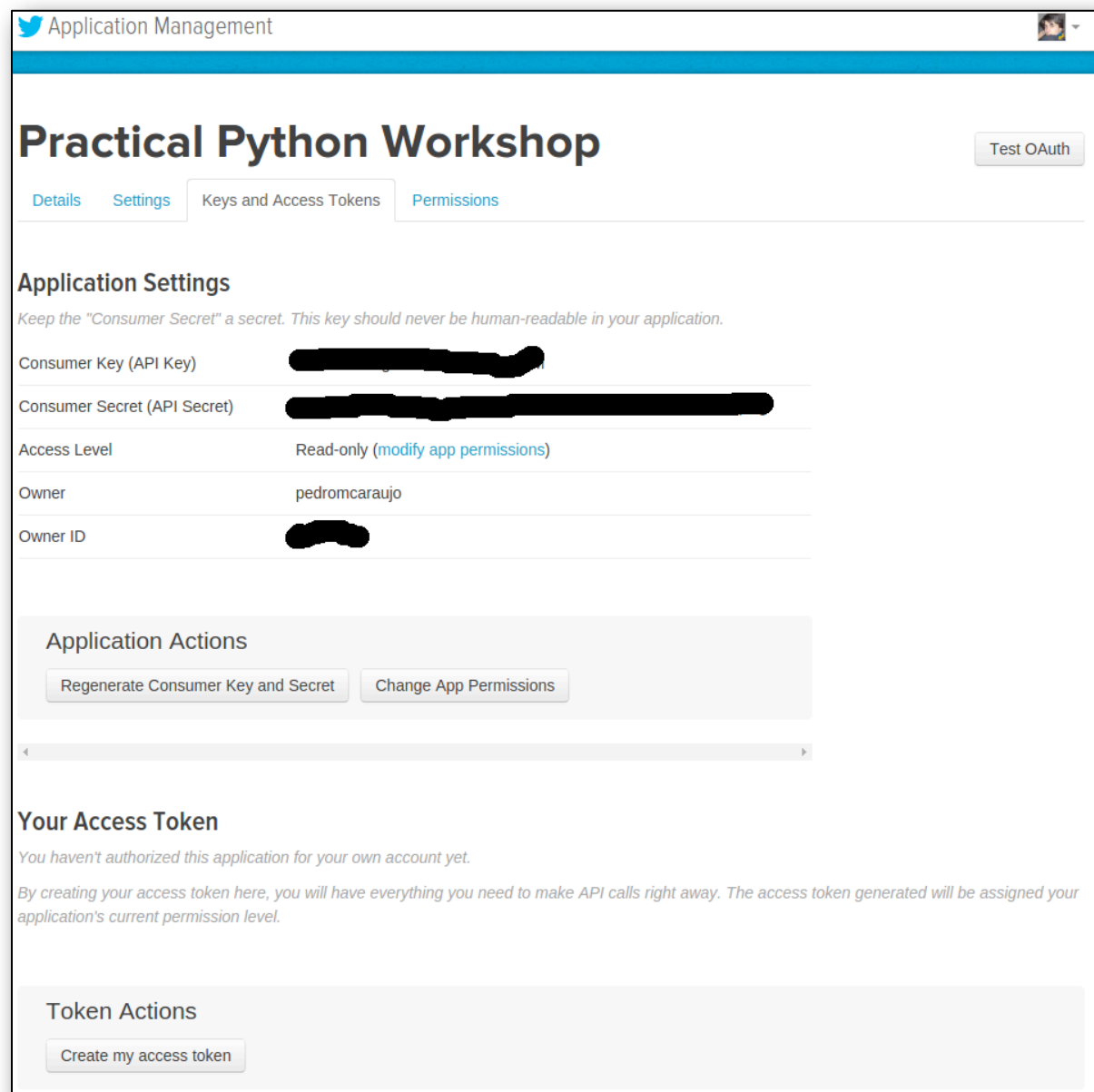
```
$ virtualenv python-workshop
(python-workshop) $ pip install tweepy
Downloading/unpacking tweepy
  Downloading tweepy-3.1.0-py2.py3-none-any.whl
Downloading/unpacking requests-oauthlib==0.4.1 (from tweepy)
  Downloading requests_oauthlib-0.4.1-py2.py3-none-any.whl
Downloading/unpacking six==1.7.3 (from tweepy)
  Downloading six-1.7.3-py2.py3-none-any.whl
Downloading/unpacking requests==2.4.3 (from tweepy)
  Downloading requests-2.4.3-py2.py3-none-any.whl (459kB): 459kB
downloaded
Downloading/unpacking oauthlib>=0.6.2 (from requests-oauthlib==0.4.1-
>tweepy)
  Downloading oauthlib-0.7.2.tar.gz (106kB): 106kB downloaded
Running setup.py
(path:/home/pma/.virtualenvs/workshop/build/oauthlib/setup.py) egg_info for
package oauthlib

Installing collected packages: tweepy, requests-oauthlib, six, requests,
oauthlib
Running setup.py install for oauthlib

Successfully installed tweepy requests-oauthlib six requests oauthlib
Cleaning up...
$
```

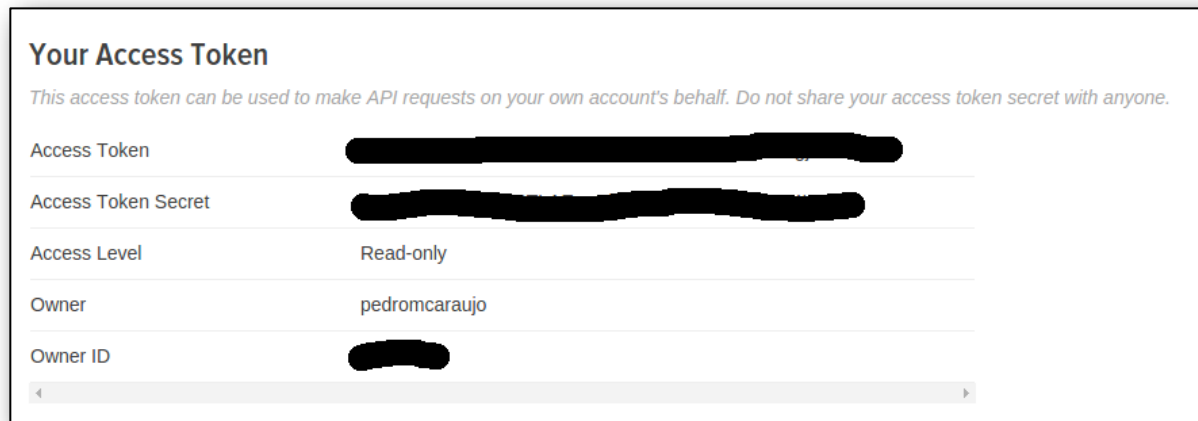
After this is installed we need to get some API keys so we can use twitter's API. To do that just go to <https://apps.twitter.com/app/new> (you'll need to be signed in to be able to register your app). Fill the form (the application website is not really necessary, you can just put anything there).

After this is done you should get sent to your application page:



The screenshot shows the Twitter Application Management interface. At the top, there's a blue header with the Twitter logo and 'Application Management'. Below this, the application name 'Practical Python Workshop' is displayed, along with a 'Test OAuth' button. A navigation bar contains links for 'Details', 'Settings', 'Keys and Access Tokens', and 'Permissions'. The 'Settings' tab is active, showing 'Application Settings'. A note states: 'Keep the "Consumer Secret" a secret. This key should never be human-readable in your application.' The settings include: 'Consumer Key (API Key)' (redacted), 'Consumer Secret (API Secret)' (redacted), 'Access Level' (Read-only, with a link to 'modify app permissions'), 'Owner' (pedromcaraujo), and 'Owner ID' (redacted). Below the settings is an 'Application Actions' section with buttons for 'Regenerate Consumer Key and Secret' and 'Change App Permissions'. A horizontal scrollbar is visible. The 'Your Access Token' section follows, with a note: 'You haven't authorized this application for your own account yet.' and another note: 'By creating your access token here, you will have everything you need to make API calls right away. The access token generated will be assigned your application's current permission level.' At the bottom is a 'Token Actions' section with a 'Create my access token' button.

Your application is created and you have your consumer key and consumer secret to be able to tell Twitter this is one making request. But now we need 2 more keys to tell twitter which user is making the request. Those keys are called "Access Tokens". To get them just click on the "Create my access token" button at the bottom of the page. After you click it you should get a new section on the bottom of the page like this one:



Awesome. At this point you all 4 keys needed to authenticate with Twitter's API:

- Consumer Key
- Consumer Secret
- Access Token
- Access Token Secret

At this point we're ready to ask anything out of Twitter. Let's start with some simple things.

Create a new python file in your preferred editor and fill the following python variables with it.

```
CONSUMER_KEY = ""
CONSUMER_SECRET = ""
ACCESS_TOKEN = ""
ACCESS_TOKEN_SECRET = ""
```

Now, with tweepy installed, let's instantiate a Twitter API object and get it ready for requests.

```
import tweepy
auth = tweepy.OAuthHandler(CONSUMER_KEY, CONSUMER_SECRET)
auth.set_access_token(ACCESS_TOKEN, ACCESS_TOKEN_SECRET)
api = tweepy.API(auth)

tweets = api.home_timeline(count=5)

for tweet in tweets:
    print "User", tweet.user.name, "tweeted:", tweet.text
```

This small piece of code will get the 5 most recent tweets in our timeline and show the user who tweeted them as well as the tweet text.

If you check what is a tweet (what type of object it is) python will tell you it's a "Status" object. That's because tweepy's developers created a class to put all the status attributes in. If you want to know everything a Status object has you have 2 main options. Either find Tweepy's source code and look at it or use the built-in dir function. You can check the dir function's documentation by running "print dir.__doc__" like you've seen earlier.

```
type(tweet)
tweepy.models.Status
dir(tweet)
['__class__',
.....
'author',
'contributors',
'coordinates',
'created_at',
'destroy',
'entities',
'favorite',
'favorite_count',
'favorited',
'geo',
'id',
'id_str',
'in_reply_to_screen_name',
'in_reply_to_status_id',
'in_reply_to_status_id_str',
'in_reply_to_user_id',
'in_reply_to_user_id_str',
'lang',
'parse',
'parse_list',
'place',
'retweet',
'retweet_count',
'retweeted',
```

```
retweets',  
'source',  
'source_url',  
'text',  
'truncated',  
'user']
```

I've cut out some of the output from `dir` (you can ignore the attributes that start with an underscore, they are mostly for the object's internal use). As you can see there's a lot of information you can get just from a single tweet.

Now I leave you to explore Twitter's API. The same way you did a `dir` on a `Status` object you can do it on anything to see their attributes. Including the "api" object we instantiated earlier. Try it. Don't forget you can use "`.__doc__`" to check what a method does (not all methods will have documentation but they're self explanatory).

Here are some fun calls you can try:

- `api.search_users(query)` - search users in the Twitter DB
- `api.get_status(tweet_id)` - get's a specific tweet by id
- `api.get_search(latitude, longitude)` - will returns tweets in the vicinity of the coordinates you sent (try: `api.geo_search("37.391933", "-122.04375")` for tweets near Mountain View)

If you want to check Tweepy's full documentation just go to:

<http://docs.tweepy.org/en/v3.5.0/>

COMPETENCES GAINED AFTER MODULE 2

- understanding basic python data types
- knowing what classes are and how to use them
- understanding exceptions and know how to handle them
- knowing how to get data from a publicly accessible API

WHAT YOU LEARNED IN THIS MODULE:

- Python flow control statements
- list, dictionaries and advanced operations on them (slicing and iterating)
- the difference between classes and instances of an object
- how Python initialises new objects
- how to override Python's built-in methods and types
- how to login to Twitter through Python
- how to get an instance's attributes using the Python shell

MODULE 3

FILES

FILES

Files are a big part of programming. We use them for a lot of things. HTML files have to be loaded when serving a web page. Some applications export files in some formats that we need to read in other applications or even we want to be the ones doing the exporting. In this module we will learn some concepts to help us understand how to use files and also some advanced ways of making use of them.

DUCK TYPING

Duck typing is a very common way of typing objects in Python. The name Duck Typing comes from the expression “If it walks like a duck, swims like a duck and quacks like a duck. It is a duck”. In programming languages this means that if an object is not of the type you desire but has the same methods then it must do the same thing. To understand this concept more in depth we’ll be using Python’s built-in StringIO object.

StringIO is a file-like object that does not save files. This is very useful, for example, when you download a file from a web service but don’t need to store it. We can put the file in a StringIO object and it will behave exactly like an actual file (because StringIO has the same methods as file objects). Contrary to file objects StringIO will only save the file’s contents to memory and not to disk (making it very fast when compared to actual files), with the downside that they are temporary (which in some situations is exactly what we need).

When initialising a file you always need to provide 2 arguments, a file path and a opening mode (the most used modes, like we used on module 1 are 'r' and 'w' for reading and writing respectively). With a StringIO we only need to instantiate one without any arguments to get an empty file. If you want to initialise it with content just pass a string as the first argument. For example, if we want to store the contents of <https://google.com/> temporarily in memory to do something with it we could do:

```
$ response = request.get("https://google.com/")
$ google_content = StringIO(response.content)
```

From now on the variable `google\_content` will behave like a file and can be passed to any library or package that expects a file. This is all due to duck-typing.

OPENING AND READING FROM FILES

Let's practice a bit opening and reading files. In this section I'll try to show some quirks about opening files like "Universal newline" and such. First thing we need is a file. We can create a new empty file on disk by doing:

```
$ f = open('/home/path/to/file/file.txt', 'w')
```

The mode 'w' indicates that we are opening the file for writing and if no file exists with the name and path provided one will be created. Note that if there is a file with the same name as the one you are trying to edit, it will be erased. If you want to append information to an existing file, use the 'a' mode. Try it.

When you are done reading the data from the files, you should close the file by calling:

```
$ f.close()
```

This will release the file and free up any system resources used by the opening of your file.

As of python 2.5, a new statement was introduced to simplify this process: [the with statement](#). This statement clarifies some code that previously would use try/finally blocks, so that it can be written in a more pythonic way. Using this, you can open a file and when you no longer use it, the file will be properly closed, even if some exceptions are raised along the way, and the system resources will be freed. Here's an example of the proper opening of a file:

```
with open('workfile', 'r') as f:
    read_data = f.read()
```

CSV FILES AND CSVREADER

Files can have many formats. One of the most common is CSV (comma separated values but you can also see TSV for tab separated values). The format of these files is very simple. The first row is either a comma separated values of headers or directly data. The [file](#) we downloaded in the first module is a CSV file. If you open the file, you can see that there is a header in the first line and the rest of the data follows.

READ

To read a csv file, you need to use the csv python module, therefore, it needs to be imported before you can use it (import csv). After that, and with an opened file, you can use the reader from the csv module to create a reader, which can iterate over all the lines in the csv file. Take a look at this example:

```
>>> import csv
>>> with open('csvfile.csv', 'rU') as f:
...     reader = csv.reader(f, delimiter=',', dialect='excel')
...     for row in reader:
...         print row
...

['street', 'city', 'zip', 'state', 'beds', 'baths', 'sq__ft', 'type',
'sale_date', 'price', 'latitude', 'longitude']

['3526 HIGH ST', 'SACRAMENTO', '95838', 'CA', '2', '1', '836',
'Residential', 'Wed May 21 00:00:00 EDT 2008', '59222', '38.631913',
'-121.434879']

['51 OMAHA CT', 'SACRAMENTO', '95823', 'CA', '3', '1', '1167',
'Residential', 'Wed May 21 00:00:00 EDT 2008', '68212', '38.478902',
'-121.431028']

['2796 BRANCH ST', 'SACRAMENTO', '95815', 'CA', '2', '1', '796',
'Residential', 'Wed May 21 00:00:00 EDT 2008', '68880', '38.618305',
'-121.443839']

['2805 JANETTE WAY', 'SACRAMENTO', '95815', 'CA', '2', '1', '852',
'Residential', 'Wed May 21 00:00:00 EDT 2008', '69307', '38.616835',
'-121.439146']

...
```

In this example, you can see that we open the sample file using the with statement, and we use the opened file in the reader function. The reader function receives some useful args, as you can see above.

The delimiter defines the column separator, in this case a comma. The dialect argument identifies a specific dialect (in this case the excel), and loads a set of parameters specific to this particular dialect. You can get the list of all registered dialects using this command:

```
>>> csv.list_dialects()
['excel-tab', 'excel']
```

There are a number of extra arguments that you can pass the reader function, that you can check out in the [csv module page](#).

Once you have the row object, you can access each column by index (row[0]) or you can use the row's iterator to your advantage and traverse the row's columns in a for cycle for example.

WRITE

Writing data to a CSV file is fairly similar to reading data. You have a writer instead of a reader and you send the rows to the writer and close the file in the end. It's as simple as that:

```
>>> import csv
>>> with open('newfile.csv', 'wb') as csvfile:
...     writer = csv.writer(csvfile, delimiter=' ',
...                           quotechar='|', quoting=csv.QUOTE_MINIMAL)
...     writer.writerow(['Spam', 'Lovely Spam', 'Wonderful Spam'])
```

Looking at the example, we can see that it's similar in many aspects to the reader, including the delimiter, and other arguments. The delimiter was already explained in the reader. As for the others, the quotechar is a one-character string used to quote fields containing special characters, such as the delimiter or quotechar, or which contain new-line characters. It defaults to ' '. The quoting argument controls when the quotes are added, in this case, or when they should be read, when we are talking about the reader. As mentioned above, more arguments exist and can be used, so you should consider taking a look to the module documentation.

SIMPLEJSON

JSON is a human readable data format that became popular in web development as an alternative to XML. It is mostly used to transmit data between client and server, but can also be used to store data. Python has a library to parse json data into python data structures:

```
>>> import json
```

So, why do we need JSON? There are other ways to store and load data in python: [Pickle](#) for example. Pickle allows the serialization and unserialization of data in python. As I said in the last sentence, the “in python” part is very important. This data is only readable by python, so it is not of much use for other system integrations... JSON in the other hand has gradually become one of the main information transmission formats, mainly in the web environment, but in many other contexts.

Generate JSON data from python

In order to generate a JSON data structure directly from python, we only need python’s default json module and the data structure we need to convert:

```
>>> import json
>>> data = {'three': 3, 'five': [1, 2, 3, 4, 5], 'two': 2, 'one': 1}
>>> json.dumps(data)
'{"one": 1, "five": [1, 2, 3, 4, 5], "three": 3, "two": 2}'
```

It’s as simple as that! You are using python after all...

Parse JSON data with python

As you are probably guessing right now, reading JSON data into python is also extremely simple:

```
>>> import json
>>> json_data = '{"one": 1, "five": [1, 2, 3, 4, 5], "three": 3, "two": 2}'
>>> json.loads(json_data)
{u'five': [1, 2, 3, 4, 5], u'three': 3, u'two': 2, u'one': 1}
```

As you can see, working with JSON is extremely simple in python.

PRACTICAL EXERCISE

Now let’s try a bigger project. In this example we need to get some sample data. What we are looking for is a file with sentences (one per line). Fortunately there’s one [here](#). As you can see, the file is a CSV file, so we already know how to process one, right?

READ FILE WITH A SENTENCE PER LINE

Ok, let’s start by reading the file, one sentence per line and store it in a list to be processed later:

```
>>> import csv
>>> data = []
>>> with open('data_file.csv', 'rU') as f:
...     reader = csv.reader(f, delimiter=',', dialect='excel')
...     for line in reader:
...
...         data.append(line)
...
>>> data[:10]
[['street', 'city', 'zip', 'state', 'beds', 'baths', 'sq__ft', 'type',
'sale_date',
'price', 'latitude', 'longitude'], ['3526 HIGH ST', 'SACRAMENTO', '95838',
'CA',
'2', '1', '836', 'Residential', 'Wed May 21 00:00:00 EDT 2008', '59222',
'38.631913', '-121.434879'], ['51 OMAHA CT', 'SACRAMENTO', '95823', 'CA',
'3', '1', '1167', 'Residential', 'Wed May 21 00:00:00 EDT 2008', '68212',
'38.478902', '-121.431028'], ['2796 BRANCH ST', 'SACRAMENTO', '95815',
'CA', '2', '1', '796', 'Residential', 'Wed May 21 00:00:00 EDT 2008',
'68880', '38.618305', '-121.443839'], ['2805 JANETTE WAY', 'SACRAMENTO',
'95815', 'CA', '2', '1', '852', 'Residential', 'Wed May 21 00:00:00 EDT
2008', '69307', '38.616835', '-121.439146'], ['6001 MCMAHON DR',
'SACRAMENTO', '95824', 'CA', '2', '1', '797', 'Residential', 'Wed May 21
00:00:00 EDT 2008', '81900', '38.51947', '-121.435768'], ['5828 PEPPERMILL
CT', 'SACRAMENTO', '95841', 'CA', '3', '1', '1122', 'Condo', 'Wed May 21
00:00:00 EDT 2008', '89921', '38.662595', '-121.327813'], ['6048 OGDEN NASH
WAY', 'SACRAMENTO', '95842', 'CA', '3', '2', '1104', 'Residential', 'Wed
May 21 00:00:00 EDT 2008', '90895', '38.681659', '-121.351705'], ['2561
19TH AVE', 'SACRAMENTO', '95820', 'CA', '3', '1', '1177', 'Residential',
'Wed May 21 00:00:00 EDT 2008', '91002', '38.535092', '-121.481367'],
['11150 TRINITY RIVER DR Unit 114', 'RANCHO CORDOVA', '95670', 'CA', '2',
'2', '941', 'Condo', 'Wed May 21 00:00:00 EDT 2008', '94905', '38.621188',
'-121.270555']]

>>>
```

Now that we have the data in a list, we can process it anyway we like. Let's move on to the next section so that we can manipulate each row and gather some data from it.

MANIPULATE AND GATHER METRICS ON EACH SENTENCE

If you had the curiosity to observe the file contents before processing it, you found that in the file header we have the column names of the file data:

```
street,city,zip,state,beds,baths,sq__ft,type,sale_date,price,latitude,longitude
```

Now, let's separate the transactions by city and by type so that we can find out how many real estate properties of each type exist in each city.

If we think about it for a bit, we have to separate the data by city and, for each one, separate the data by type:

```
example = {
  'city_1': {
    'type_1': [property1, property2, property3],
    'type_2': [property10, property22, property12],
  },
  'city_2': {
    'type_1': [property5, property7, property8]
  },
}
```

This is an example of a data structure that can handle our data, you can think of other ways to store the data, as long as you can get the statistical data requested above.

So let's see how can we process the data in order to generate this structure:

```
>>> processed = {}
>>> for row in data:
...     city = row[1]
...     type = row[7]
...     if processed.has_key(city):
...         pr_city = processed[city]
...         pr_type = pr_city.get(type, [])
...         pr_type.append(row)
...         processed[city][type] = pr_type
...     else:
...         processed[city] = {type: [row]}
...

>>> processed['ANTELOPE']
{'Residential': [['3828 BLACKFOOT WAY', 'ANTELOPE', '95843', 'CA', '3', '2',
'1088', 'Residential', 'Wed May 21 00:00:00 EDT 2008', '126640', '38.70974',
'-121.37377'], ['5708 RIDGEPOINT DR', 'ANTELOPE', '95843', 'CA', '2', '2',
'1043', 'Residential', 'Wed May 21 00:00:00 EDT 2008', '161250', '38.72027',
'-121.331555'], ['4844 CLYDEBANK WAY', 'ANTELOPE', '95843', 'CA', '3', '2',
'1215', 'Residential', 'Wed May 21 00:00:00 EDT 2008', '182716', '38.714609',
'-121.347887'], ['7895 CABER WAY', 'ANTELOPE', '95843', 'CA', '3', '2', '1362',
'Residential', 'Wed May 21 00:00:00 EDT 2008', '194818', '38.711279',
'-121.393449'], ['7837 ABBINGTON WAY', 'ANTELOPE', '95843', 'CA', '4', '2',
'1830', 'Residential', 'Wed May 21 00:00:00 EDT 2008', '387731', '38.709873',
'-121.339472'], ['3228 BAGGAN CT', 'ANTELOPE', '95843', 'CA', '3', '2', '1392',
'Residential', 'Tue May 20 00:00:00 EDT 2008', '165000', '38.715346',
'-121.388163'], ['7863 CRESTLEIGH CT', 'ANTELOPE', '95843', 'CA', '2', '2',
'1007', 'Residential', 'Tue May 20 00:00:00 EDT 2008', '180000', '38.710889',
'-121.358876'], ['4437 MITCHUM CT', 'ANTELOPE', '95843', 'CA', '3', '2', '1393',
'Residential', 'Tue May 20 00:00:00 EDT 2008', '200000', '38.704407',
'-121.36113'], ['5312 MARBURY WAY', 'ANTELOPE', '95843', 'CA', '3', '2', '1574',
'Residential', 'Tue May 20 00:00:00 EDT 2008', '255000', '38.710221',
'-121.341651'], ['5712 MELBURY CIR', 'ANTELOPE', '95843', 'CA', '3', '2', '1567',
'Residential', 'Tue May 20 00:00:00 EDT 2008', '261000', '38.705849',
'-121.334701'], ['8108 FILIFERA WAY', 'ANTELOPE', '95843', 'CA', '4', '3',
'1768', 'Residential', 'Tue May 20 00:00:00 EDT 2008', '265000', '38.717042',
'-121.35468'], ['3318 DAVIDSON DR', 'ANTELOPE', '95843', 'CA', '3', '1', '988',
'Residential', 'Mon May 19 00:00:00 EDT 2008', '223139', '38.705753',
```

```
'-121.35468'], ['3318 DAVIDSON DR', 'ANTELOPE', '95843', 'CA', '3', '1', '988',
'Residential', 'Mon May 19 00:00:00 EDT 2008', '223139', '38.705753',
'-121.388917'], ['4508 OLD DAIRY DR', 'ANTELOPE', '95843', 'CA', '4', '3',
'2026', 'Residential', 'Mon May 19 00:00:00 EDT 2008', '231200', '38.72286',
'-121.358939'], ['8721 SPRUCE RIDGE WAY', 'ANTELOPE', '95843', 'CA', '3', '2',
'1187', 'Residential', 'Mon May 19 00:00:00 EDT 2008', '234000', '38.727657',
'-121.391028'], ['3305 RIO ROCA CT', 'ANTELOPE', '95843', 'CA', '4', '3', '2652',
'Residential', 'Mon May 19 00:00:00 EDT 2008', '239700', '38.725079',
'-121.387698'], ['5308 MARBURY WAY', 'ANTELOPE', '95843', 'CA', '3', '2', '1830',
'Residential', 'Mon May 19 00:00:00 EDT 2008', '254172', '38.710221',
'-121.341707'], ['4712 PISMO BEACH DR', 'ANTELOPE', '95843', 'CA', '5', '3',
'2346', 'Residential', 'Mon May 19 00:00:00 EDT 2008', '320000', '38.707705',
'-121.354153'], ['4741 PACIFIC PARK DR', 'ANTELOPE', '95843', 'CA', '5', '3',
'2347', 'Residential', 'Mon May 19 00:00:00 EDT 2008', '325000', '38.709299',
'-121.353056'], ['3361 ALDER CANYON WAY', 'ANTELOPE', '95843', 'CA', '4', '3',
'2085', 'Residential', 'Mon May 19 00:00:00 EDT 2008', '408431', '38.727649',
'-121.385656'], ['3536 SUN MAIDEN WAY', 'ANTELOPE', '95843', 'CA', '3', '2',
'1711', 'Residential', 'Fri May 16 00:00:00 EDT 2008', '161500', '38.70968',
'-121.382328'], ['4008 GREY LIVERY WAY', 'ANTELOPE', '95843', 'CA', '3', '2',
'1669', 'Residential', 'Fri May 16 00:00:00 EDT 2008', '168750', '38.71846',
'-121.370862'], ['8716 LONGSPUR WAY', 'ANTELOPE', '95843', 'CA', '3', '2',
'1479', 'Residential', 'Fri May 16 00:00:00 EDT 2008', '205000', '38.724083',
'-121.3584'], ['7901 GAZELLE TRAIL WAY', 'ANTELOPE', '95843', 'CA', '4', '2',
'1953', 'Residential', 'Fri May 16 00:00:00 EDT 2008', '207744', '38.71174',
'-121.342675'], ['4085 COUNTRY DR', 'ANTELOPE', '95843', 'CA', '4', '3', '1915',
'Residential', 'Fri May 16 00:00:00 EDT 2008', '240000', '38.706209',
'-121.369509'], ['8316 NORTHAM DR', 'ANTELOPE', '95843', 'CA', '3', '2', '1235',
'Residential', 'Fri May 16 00:00:00 EDT 2008', '246544', '38.720767',
'-121.376678'], ['4240 WINJE DR', 'ANTELOPE', '95843', 'CA', '4', '2', '2504',
'Residential', 'Fri May 16 00:00:00 EDT 2008', '246750', '38.70884',
'-121.359559'], ['4636 TEAL BAY CT', 'ANTELOPE', '95843', 'CA', '4', '2', '2160',
'Residential', 'Fri May 16 00:00:00 EDT 2008', '290000', '38.704554',
'-121.354753'], ['7921 DOE TRAIL WAY', 'ANTELOPE', '95843', 'CA', '5', '3',
'3134', 'Residential', 'Fri May 16 00:00:00 EDT 2008', '315000', '38.711927',
'-121.343608'], ['4509 WINJE DR', 'ANTELOPE', '95843', 'CA', '3', '2', '2960',
'Residential', 'Fri May 16 00:00:00 EDT 2008', '350000', '38.709513',
```



```
'-121.359357'], ['3604 KODIAK WAY', 'ANTELOPE', '95843', 'CA', '3', '2', '1206',  
'Residential', 'Thu May 15 00:00:00 EDT 2008', '142000', '38.706175',  
'-121.379776'], ['8636 LONGSPUR WAY', 'ANTELOPE', '95843', 'CA', '3', '2',  
'1670', 'Residential', 'Thu May 15 00:00:00 EDT 2008', '157296', '38.725873',  
'-121.35856'], ['8428 MISTY PASS WAY', 'ANTELOPE', '95843', 'CA', '3', '2',  
'1517', 'Residential', 'Thu May 15 00:00:00 EDT 2008', '212000', '38.722959',  
'-121.347115']]}, 'Condo': [['8020 WALERGA RD', 'ANTELOPE', '95843', 'CA', '2',  
'2', '836', 'Condo', 'Mon May 19 00:00:00 EDT 2008', '115000', '38.71607',  
'-121.364468']]}
```

Now we have the data in the format that we want, but it is still not very readable... Let's make a function to pretty print the data in a more human way:

```
>>> def pretty_print_data(data):  
...     for city in data:  
...         print "City: %s" % (city,)   
...         for type in data[city]:  
...             print "  Type: %s - %d" % (type, len(data[city][type]))
```

Now, let's try it and see some sample output:

```
>>> pretty_print_data(processed)  
City: ORANGEVALE  
  Type: Residential - 11  
City: CITRUS HEIGHTS  
  Type: Residential - 32  
  Type: Condo - 2  
  Type: Multi-Family - 1  
City: SACRAMENTO  
  Type: Residential - 402  
  Type: Condo - 27  
  Type: Multi-Family - 10  
...
```

OUTPUT A FILE WITH THE METRICS OBTAINED

We now have the statistical data. But what can we do with it? Let's save it in a file, using the JSON format, so that it can be passed to other applications:

HAKING

```
>>> import json
>>> with open('statistics.json', 'wb') as f:
...     json_data = json.dumps(processed)
...     f.write(json_data)
...
>>>
```

And that's it! Try to read the data from the newly created JSON file, so that you get the hang of it...

COMPETENCES GAINED AFTER MODULE 3:

- understanding the concept of duck typing
- understanding how files work
- understanding the csv python module to read/write csv files
- understanding the json python module and how to use it
- use this modules in a real life example to extract, process and save data

MODULE 4

PRACTICAL PROJECT - WEATHER FORECAST!

DESCRIPTION

In this module, we are going to apply all the knowledge from the previous chapters in order to implement a Python module to read data from an API, process the information and display it, using Python plotting library, in a friendly way.

As we should do in any development, we have to define exactly what our module does:

- Read data from an API (<http://openweathermap.org>)
- Save the raw data in a file for safekeeping
- Transform the data, so that it can be fed to the plot module
- Plot a graph with the weather forecast for the day

GET INFORMATION FROM API

We are going to process the information from the Open Weather Map API. Let's use this URL to get the forecast for a group of cities:

<http://api.openweathermap.org/data/2.5/box/city?bbox=12,32,15,37,10&cluster=yes>

Now we need a function to get the json data from this URL. For this we will use the requests library. This library is not a Python built-in module so you have to install it. You still remember how to install packages, using pip? (If not, check module 2)

```
$ pip install requests
```

Now that we have all the dependencies we need, let's create a simple Python file that will hold all our code for this module. Let's call it module4.py.

Now we have to import our request dependencies and create a function to get the forecast data in json. Try to do this alone before looking at the example:

```
import requests
def get_forecast(url):
    """ Return the forecast data in json
    """
    r = requests.get(url)
    return r.json()
```

If you print the result for the URL above, you get something like this:

HAKING

```
{u'list': [{u'clouds': {u'all': 0}, u'name': u'Yafran', u'coord': {u'lat': 32.06329, u'lon': 12.52859},
u'weather': [{u'main': u'Clear', u'id': 800, u'icon': u'01d',
u'description': u'Sky is Clear'}], u'dt':
1437555483, u'main': {u'temp': 31.92, u'grnd_level': 958.15, u'temp_max': 31.923,
u'sea_level': 1028.38, u'humidity': 29, u'pressure': 958.15, u'temp_min': 31.923}, u'id':
2208791, u'wind': {u'speed': 1.81, u'deg': 212.001}}, {u'clouds': {u'all': 8}, u'name':
u'Zuwarah', u'coord': {u'lat': 32.931198, u'lon': 12.08199}, u'weather':
[{u'main': u'Clear',
u'id': 800, u'icon': u'02d', u'description': u'Sky is Clear'}], u'dt':
1437555483, u'main': {u'temp':
26.62, u'grnd_level': 1027.37, u'temp_max': 26.623, u'sea_level':
...

u'main': {u'pressure': 1013, u'temp_min': 32, u'temp_max': 32, u'temp': 32,
u'humidity': 46}
u'id': 2524119, u'wind': {u'speed': 1, u'deg': 110}}, {u'clouds': {u'all': 0}, u'name': u'Rosolini',
u'coord': {u'lat': 36.824242, u'lon': 14.94779}, u'weather': [{u'main':
u'Clear', u'id': 800,
u'icon': u'01d', u'description': u'Sky is Clear'}], u'dt': 1437555556,
u'main': {u'temp': 27.82,
u'grnd_level': 1024.46, u'temp_max': 27.823, u'sea_level': 1026.39,
u'humidity': 93,
u'pressure': 1024.46, u'temp_min': 27.823}, u'id': 2523581, u'wind':
{u'speed': 1.61, u'deg':
277.501}}], u'cnt': 15, u'calctime': 0.0059, u'cod': u'200'}
```

Now, save the data in a file with a date-time in the name (Ex: forecast-2015522.json). You still remember how to do it, right?

Now, let's break down the json structure. You can use any online tool to “pretty print” the data you just received, so that you can better understand its current structure:

```
{
  "message": "accurate",
  "cod": "200",
  "count": 10,
  "list": [
    {
      "id": 495260,
      "name": "Shcherbinka",
      "coord": {
        "lon": 37.559719,
        "lat": 55.499722
      },
      "main": {
        "temp": 294.25,
        "pressure": 1009,
        "humidity": 64,
        "temp_min": 293.15,
        "temp_max": 296.15
      },
      "dt": 1437557440,
      "wind": {
        "speed": 6,
        "deg": 280
      },
      "sys": {
        "country": ""
      },
      "clouds": {
        "all": 75
      },
      "weather": [
        {
          "id": 520,
```

```
"main": "Rain",  
    "description": "light intensity shower rain",  
    "icon": "09d"  
}  
]  
,  
.....  
]  
}
```

DATA TRANSFORMATION

Let's think a little about the data structure that we need: we want to present, for each city, a bar chart comparing the temperature and humidity for each of them.

In order to draw a bar chart, we need the information in lists, ordered. So, let's define the lists that we need:

- cities: the list of city names
- temperatures: the list of the temperatures, maintaining the same order of the cities list
- humidities: the list of humidities, maintaining the same order of the cities also

Create a function that receives the raw json data from the API, processes it and returns a dict with the information in the list above. Again, try to do it yourself before looking at the next example:

```
def process_data(data):  
    """ Return data to be used by the plot lib  
    """  
    info = {  
        'cities': [],  
        'temperatures': [],  
        'humidities': [],  
    }  
    cities = data['list']  
    for city in cities:  
        main_data = city['main']  
        info['cities'].append(city['name'])  
        info['temperatures'].append(main_data['temp'])  
        info['humidities'].append(main_data['humidity'])  
  
    return info
```

This will return something like the following:

```
{'humidities': [22, 60, 99, 27, 32, 27, 22, 37, 32, 32, 55, 62, 93, 74,  
98], 'cities':  
[u'Yafran', u'Zuwarah', u'Sabratah', u'Gharyan', u'Zawiya', u'Tripoli',  
u'Tarhuna',  
u'Masallatah', u'Al Khums', u'Zlitan', u'Birkirkara', u'Ragusa',  
u'Pozzallo', u'Modica',  
u'Rosolini'], 'temperatures': [35.31, 30.31, 26.36, 35.63, 35.73, 35.63,  
35.91,  
33.88, 34.51, 34.51, 31.4, 30.1, 27.51, 31, 27.43]}
```

PLOTTING THE DATA

In order to visually render our data, we will use an external library: Matplotlib. You can install it the same way you installed requests, or check other installation formats [here](#).

Once you have installed the package, you can read a little of the documentation to try to plot the data yourself. Draw a barchart with the city names in the X axis and the humidity and temperature values in the Y axis.

So, let's make a function to do all that work for us:

```
def show_plot(data):  
    """ Compute and plot the bar chart  
    """  
  
    cities = tuple(data['cities'])  
    temperatures = tuple(data['temperatures'])  
    humidities = tuple(data['humidities'])  
    N = len(cities)  
  
    # Define the width of each bar, and create a list of positions  
    # that will be used to place each bar in the chart  
    ind = np.arange(N) # the x locations for the groups  
    width = 0.35       # the width of the bars  
  
    _, ax = plt.subplots()  
    rects1 = ax.bar(ind, temperatures, width, color='r')  
    rects2 = ax.bar(ind+width, humidities, width, color='y')  
    # Show the bar chart  
    plt.show()
```

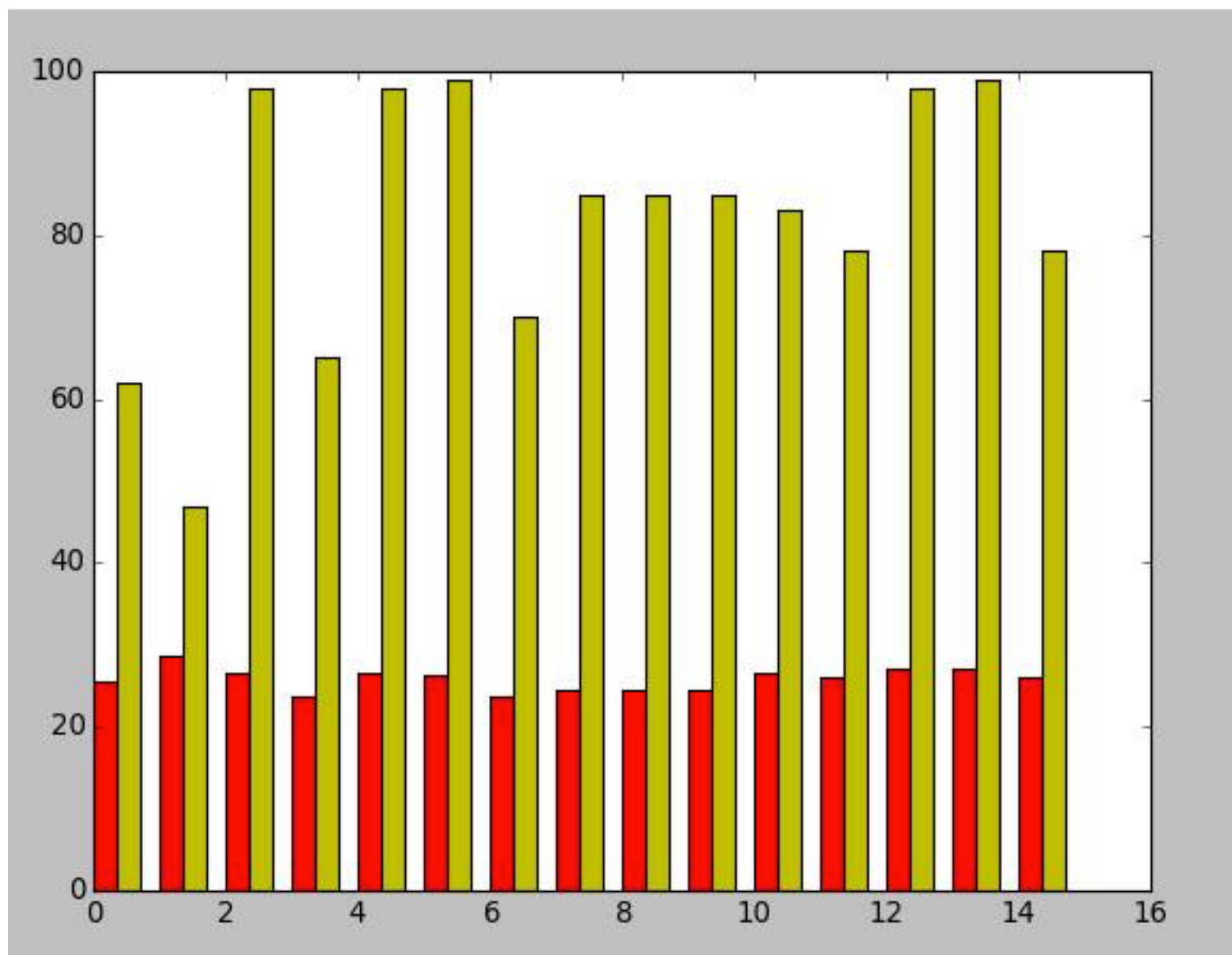
Let's try to break down this function a bit. I will explain each section of the function, so that you can better understand what everything does:

```
_, ax = plt.subplots()
```

In this case, the underscore indicates that the first argument returned by the function is being deliberately ignored. You can assign the value to a variable, but in this case it would never be used...

Using the ax (Axes object - check the [documentation](#)), we create a bar for the temperatures and other for the humidities (check the examples for more options too).

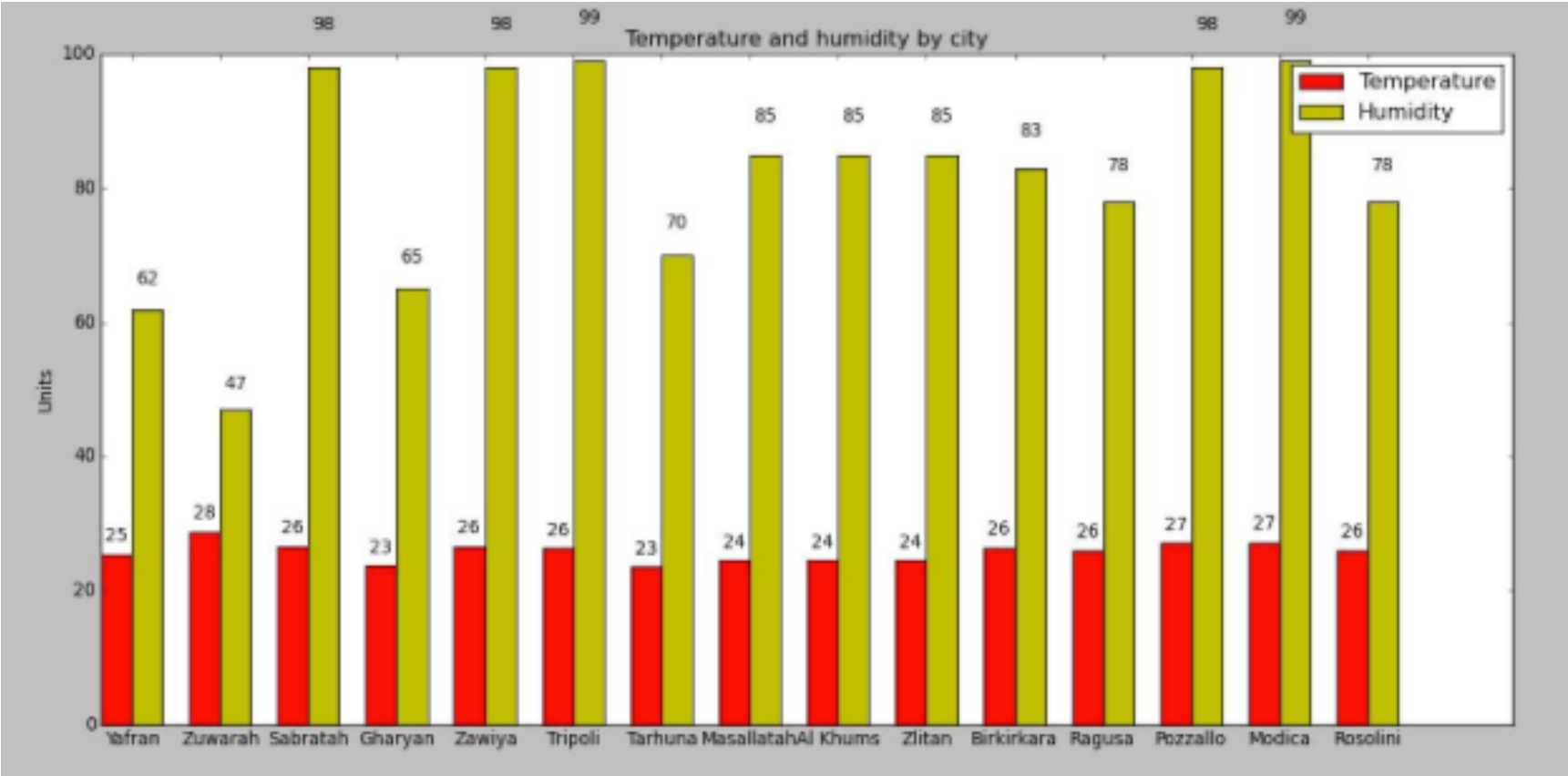
After that we only have to display the chart, which should be something like this (if you want the full source code for this graph generation you can get it [here](#)):



Now this bar chart is too simple and not that informative... You should play a bit with these options to create a chart that is actually useful:

- `ax.set_ylabel`
- `ax.set_title`
- `ax.set_xticks`
- `ax.set_xticklabels`
- `ax.legend`

Try to create this chart:



Notice that we have the value of each column above it and labels for the cities. There is also a legend in the upper right corner and a title for the graph, which is much more informative than the previous, don't you agree?

COMPETENCES GAINED AFTER MODULE 4

- Get data from an external API
- Transform data to suit your needs
- Work with the Python plotting

INSTRUCTORS:

RUI SILVA

RUI SILVA IS A COMPUTER SCIENTIST FROM PORTUGAL. [HTTP://ALFASITE.ORG/](http://alfasite.org/)

MY NAME IS RUI SILVA AND I'M A PYTHON DEVELOPER WHO LOVES OPEN SOURCE. I STARTED WORKING AS A FREELANCER IN 2008, WHILE I FINISHED MY GRADUATION IN COMPUTER SCIENCE IN UNIVERSIDADE DO MINHO. AFTER MY GRADUATION, I STARTED PURSUING A MASTER'S DEGREE, CHOOSING THE FIELD OF PARALLEL COMPUTATION AND MOBILE AND UBIQUITOUS COMPUTING. I ENDED UP ONLY FINISHING THE MOBILE AND UBIQUITOUS COMPUTING COURSE. IN MY 3 YEARS OF FREELANCING, I WORKED MOSTLY WITH PYTHON, DEVELOPING DJANGO WEBSITES, DRUPAL WEBSITES AND SOME MAGENTO STORES. I ALSO HAD TO DO SOME SYSTEM ADMINISTRATION. AFTER THAT, I STARTED WORKING IN EUROTUX INFORMÁTICA, S.A. WHERE I DEVELOP WEBSITES USING PLONE, DJANGO AND DRUPAL. I'M ALSO AN IOS DEVELOPER AND SOMETIMES I PERFORM SOME SYSTEM ADMINISTRATION TASKS. BESIDES MY JOB, I WORK AS A FREELANCER USING MAINLY DJANGO AND OTHER PYTHON FRAMEWORKS.

PEDRO ARAUJO

PEDRO ARAUJO WAS GRADUATED FROM COMPUTER SCIENCE OF UNIVERSIDADE DO MINHO, AFTER THIS, HE HAD PURSUED HIS MASTERS DEGREE IN INFORMATICS COURSE CHOOSING PARALLEL AND DISTRIBUTED COMPUTING AND FORMAL METHODS. PEDRO ARAUJO HAS DIVERSE PROFESSIONAL EXPERIENCES AS HE FIRST WORKED AT EUROTUX, S.A. WHERE HE DEVELOPED WEB SITES USING PLONE AND ZOPE IN ADDITION TO SOME SYSTEM ADMINISTRATION FUNCTIONS LIKE DEVELOPING PLUGINS FOR RAILS AND SEVERAL ANDROID DEVELOPMENT PERFORMED ON A PERSONAL BASE. THEN, HE MOVED TO WEDO TECHNOLOGIES, A COMPANY THAT FOCUSES IN REVENUE ASSURANCE. THERE, HE MAINLY WORKED ON JAVA DEVELOPMENTS, SOLARIS AND LINUX SERVERS. PRESENTLY, HE IS WORKING IN NETWORK LOCUM, A PROMISING STARTUP AND TWO ADDITIONAL PROJECTS, NOVAWEB AND ATTENTIVE. ALL IN ALL, HE IS A COMPLETE SUPPORTER OF OPEN SOURCE AND HIS MOST PREFERRED PROGRAMMING LANGUAGE IS PYTHON.